

Mini-project 6.3: Applying supervised learning to predict student dropout

Rodrigo Grandy, PhD

2025-05-25

Table of contents

1	Introduction	2
2	Objectives	2
3	Methods	2
4	Results	5
5	Conclusions	7
6	References	9

1 Introduction

In this project, we will explore the application of supervised learning techniques, such as XGBoost and Neural Networks, for analyzing structured data. XGBoost, a gradient-boosted decision tree algorithm, is ideal for handling structured data (Chen & Guestrin, 2016). Neural Networks are flexible deep learning algorithms that can capture complex patterns and relationships within data without heavily relying on its structure (LeCun, Bengio, & Hinton, 2015).

We will preprocess the data through steps such as encoding, scaling, and feature engineering. Following this, we will evaluate and fine-tune both models using robust metrics, including ROC-AUC, F1-score, and precision-recall. This comparison aims to reveal which approach best balances performance and interpretability in educational applications.

2 Objectives

We will develop a dropout prediction system using three progressive datasets: - Core student profiles (demographics, course info) - Engagement metrics (authorized/unauthorized absences) - Academic performance (module pass/fail rates)

Using XGBoost and Neural Networks, we'll identify at-risk students while testing how predictive power improves with additional data layers.

The project will demonstrate how machine learning is influenced by the quality of the data used to train the model. We will also experience the impact of feature engineering and hyperparameter tuning on model performance.

Our final report will summarize our observations and the differences in performance observed with each method in each phase of the project.

3 Methods

All work in this report was implemented in Python 3.12.3. Data Preprocessing: Datasets underwent preprocessing using pandas (v2.2.3). Irrelevant columns and high cardinality features were removed. "Date of Birth" was transformed into

“Age.” Columns with >50% missing data were dropped. Ordinal encoding was applied to “CourseLevel” using scikit-learn (v1.6.1) (Pedregosa et al., 2011). Absence counts were binned into categories. One-hot encoding was used for other categorical features. Missing values were addressed by removing observations when the fraction of NAs was under 2%. For Stage 3, “RateModuleSuccess” was computed. NumPy (v1.26.4) was used for numerical computations.

Table 1: XGBoost Model Performance before and after hyperparameter tuning

Model	Stage	ROC AUC	Precision (Class 0)	Recall (Class 0)	F1 Score (Class 0)	Precision (Class 1)	Recall (Class 1)	F1 Score (Class 1)
Base	1	0.853	0.626	0.422	0.504	0.904	0.956	0.929
Tuned	1	0.86	0.627	0.403	0.491	0.901	0.958	0.929
Base	2	0.895	0.687	0.486	0.569	0.916	0.962	0.939
Tuned	2	0.893	0.712	0.476	0.57	0.915	0.967	0.94
Base	3	0.993	0.893	0.911	0.902	0.985	0.981	0.983
Tuned	3	0.994	0.889	0.917	0.903	0.986	0.98	0.983

Table 2: Neural Network Performance before and after hyperparameter tuning

Model	Stage	Model.1	ROC-AUC	F1	Precision	Recall
Base	1	Model_6_4L_64U	0.854	0.929	0.897	0.963
Tuned	1	Model_7_adamO_16BS_0.001LR_0.2D	0.853	0.929	0.895	0.965
Base	2	Model_1_2L_32U	0.887	0.937	0.91	0.965
Tuned	2	Model_7_adamO_16BS_0.001LR_0.2D	0.888	0.939	0.923	0.955
Base	3	Model_5_2L_32U	0.985	0.972	0.98	0.965
Tuned	3	do:0.2 lr:0.001 opt:adam bs:32 L1:0.001 L2:0.001	0.984	0.972	0.986	0.958

Model Development: XGBoost (v3.0.1) models were initially trained without tuning (Chen & Guestrin, 2016). Hyperparameter tuning used GridSearchCV with 3-fold cross-validation, optimizing for ROC AUC. Key hyperparameters included learning_rate, max_depth, and n_estimators. Neural networks were built using TensorFlow (v2.19.0) and Keras (v3.10.0) with 1-3 hidden layers (16-128 units each), ReLU activation, and Softmax output. Dropout (0.2-0.3) and L1/L2 regularization were applied. Models used Adam or RMSProp optimizers (learning rate 0.01-0.001) and categorical sparse crossentropy loss. Early stopping (patience=3) was implemented. Deterministic batch shuffling ensured reproducibility. Batch sizes of 16 and 32 were tested. Model complexity was reduced to prevent overfitting and improve generalization.

A



B



B



Figure 1: Confusion Matrices for the best XGBoost Models at each stage: (A) XGBoost stage 1, (B) XGBoost stage 2, (C) XGBoost stage 3

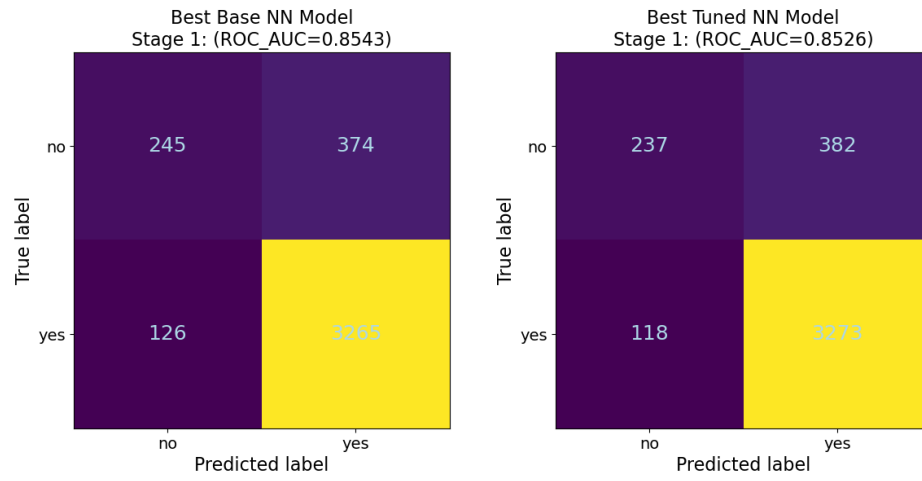
Evaluation Framework: Models used 80:20 stratified train-test splits with StandardScaler normalization. Metrics included ROC-AUC, precision, recall, and F1-score (weighted average). Class-specific metrics addressed imbalance. Confusion matrices and SHAP values enhanced interpretability. ROC-AUC on validation sets selected the best model. Class imbalance was addressed by focusing on precision-recall tradeoffs for minority classes.

4 Results

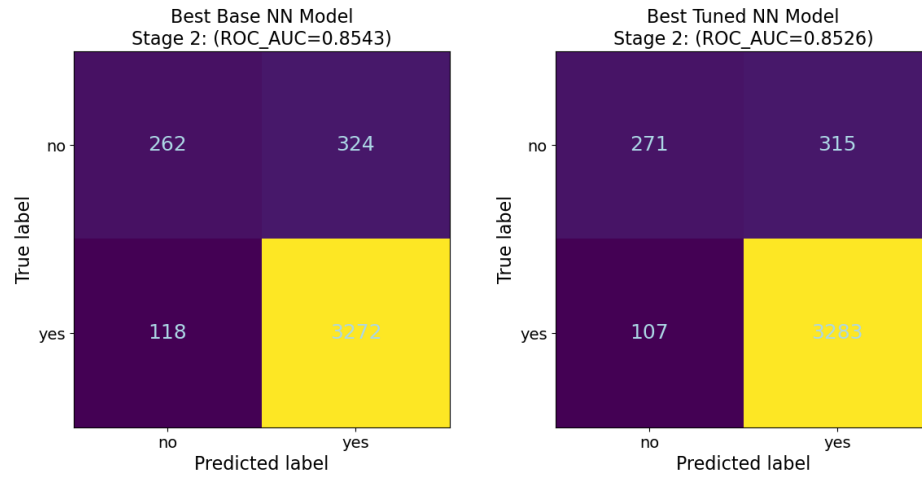
The transition from Stage 1 to Stage 2 significantly improved model performance, driven by the inclusion of student absence data (See Notebook). This data enabled XGBoost to enhance accuracy and discrimination, particularly for at-risk students (minority class 0). The base XGBoost model's ROC AUC increased from 0.85 (Stage 1) to 0.89 (Stage 2), with engineered features like `Unauthorised_Absence_Ordinal` and `Autorised_Absence_Ordinal` improving minority class recall. A tuned Stage 2 XGBoost model achieved a 0.89 ROC AUC on the testing dataset, a notable improvement from Stage 1's 0.86 (See table 1).

Stage 3's introduction of `RateModuleSuccess` further enhanced prediction, with base XGBoost models reaching a 0.993 ROC AUC on the testing dataset. Interestingly, hyperparameter tuning at this stage yielded minimal improvements, achieving a 0.994 ROC AUC (See table 1). XGBoost models showed significant improvement across stages, particularly in detecting at-risk students. The inclusion of absence data and `RateModuleSuccess` had a greater impact on model performance than tuning, indicating these features were primary drivers of enhancement. Stage 3's new feature also mitigated class imbalance issues (See table 1, figure 1, and notebook). Neural Network (NN) models showed promising results but were slightly less effective than XGBoost models at every stage. The best NN model at Stage 2 achieved a 0.888 ROC-AUC (tuned), lower than the corresponding XGBoost model. Similar differences were observed for base models (See tables 1 and 2). NN models also improved substantially in Stage 3, achieving high accuracy and specificity. As with XGBoost, fine-tuning had minimal impact on NN performance, underscoring the importance of feature engineering in addressing class imbalance and improving model performance (figures 1 and 2).

A



B



B

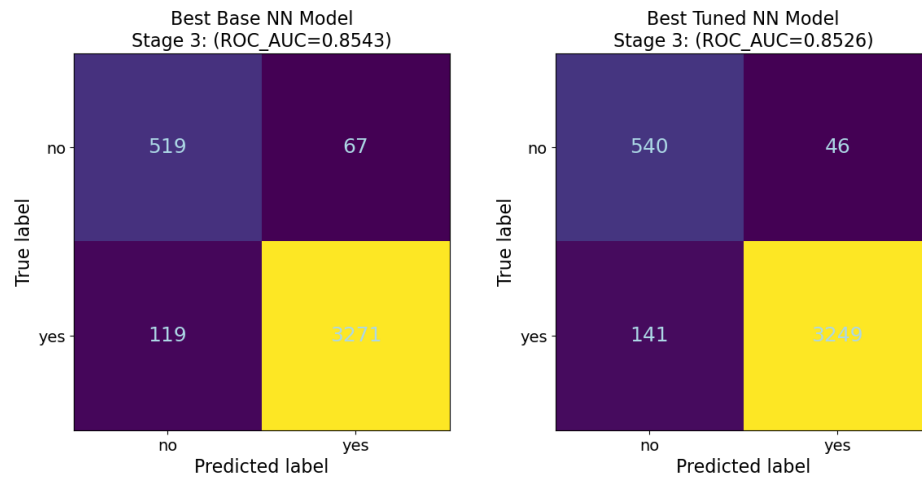


Figure 2: Confusion Matrices for the best Neural Network (NN) Models at each stage: (A) NN stage 1, (B) NN stage 2, (C) NN stage 3

Initial NN architectures exhibited overfitting, addressed through L1/L2 regularization and parameter optimization (See Notebook). The optimized NN model features a single hidden layer with 32 units, ReLU activation, and L1/L2 regularization, showing rapid convergence and minimal overfitting (See figure 3). This simplified architecture effectively addressed overfitting while maintaining high accuracy. The model's performance suggests complex architectures may be unnecessary for this dataset, though this approach may limit capture of extremely complex non-linear relationships and could be sensitive to feature quality. Overall, both XGBoost and NN models demonstrated the critical role of feature engineering in improving predictive performance, with XGBoost slightly outperforming NN models across all stages.

5 Conclusions

The project aimed to familiarize us with the methods used to construct, implement, and optimize machine learning algorithms for predicting or regressing labeled data. Both XGBoost and Neural Network models were explored, focusing on data enrichment, feature engineering, and model optimization.

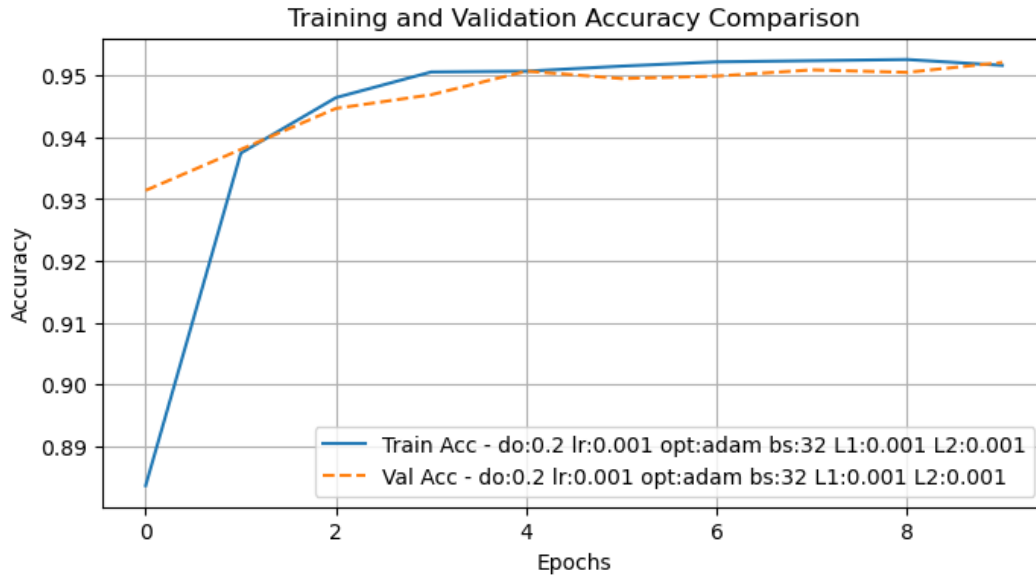
From this study, these are some of the major insights:

- Data enrichment significantly improved model performance.
- The “RateModuleSuccess” feature was highly effective but may impact model generalization.
- XGBoost outperformed neural networks, suggesting tree-based methods are more suitable for this structured dataset.
- Hyperparameter tuning provided marginal improvements compared to data representation.
- Class imbalance affected dropout prediction accuracy.
- Models’ applicability is limited to the specific dataset used.
- The most predictive features are highly correlated with dropout rates, questioning their true predictive power.

Overall, the project highlighted the importance of feature engineering and data quality in machine learning model development for educational data.

Note: Due to words limit, methodology and results analysis are limited.

A



B

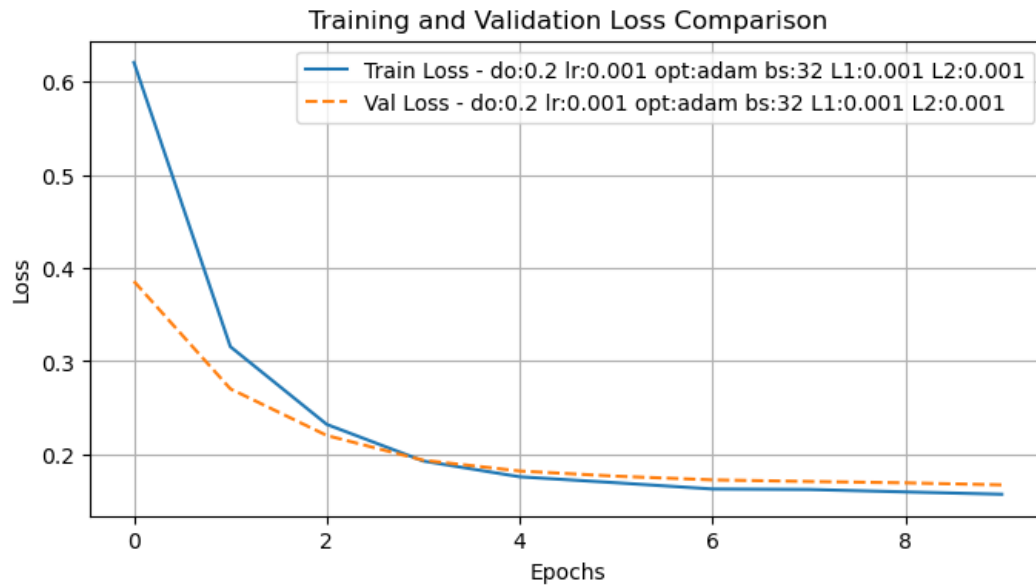


Figure 3: A neural network with one hidden layer consisting of 32 units was selected as the final model, and its learning curve is shown. (A) Training and Validation accuracy curve (B) Training and Validation loss curve. do: dropout, lr: learning rate, opt: optimizer, bs: batch size, L1: L1 regularization, L2: L2 regularization

6 References

1. Notebook: https://github.com/rgrandym/data_science_cambridge/tree/main/course/reports/course_2/mp_6.3#:~:text=Grandy_Rodrigo_CAM_C201_Week_6_Mini_project.ipynb
2. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 785-794).
3. LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436-444.
4. GitHub Repository: XGBoost - The official source for the XGBoost library, which includes installation instructions, documentation, and examples to get started.
5. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830. This paper describes the design and usage of scikit-learn for implementing machine learning algorithms in Python, emphasizing its usability and performance
6. Official Documentation: scikit-learn - Comprehensive documentation including tutorials, user guides, and the API reference for all the features provided by scikit-learn.
7. Official Documentation: TensorFlow - The primary resource for TensorFlow, offering guides, tutorials, and API references for building machine learning and neural network models.
8. Official Documentation: Keras - This documentation serves as a valuable resource for understanding how to utilize Keras for deep learning, including multiple APIs and support for various backends.